

Assignment Verification Checklist

Project: TelcoBot — AI Customer Support Chatbot for TelcoMax

Verified: May 29, 2026

1. README.md — Required Sections

#	Section	Status	Location (Line)	Notes
1	Project Title	✓	L1	“📡 TelcoBot — AI-Powered Customer Support Chatbot for TelcoMax”
2	Selected Use Case	✓	L7	Telecom customer support — billing, plans, tech, usage, account
3	Tools Used (list all)	✓	L21	Table listing OpenAI, LangChain, LangGraph, Gradio, ChromaDB, SQLite, LangSmith
4	APIs Integrated	✓	L35	OpenAI Chat Completions, OpenAI Embeddings, LangSmith API
5	LangGraph Workflow Explanation	✓	L43	Mermaid diagram + step-by-step breakdown of classifier → router → agents → tools → END
6	Memory Implementation	✓	L97	Explains MemorySaver checkpoint, thread IDs, context continuity, anaphora resolution
7	How to Run the Application	✓	L152	Prerequisites, installation steps, env setup, run command, LangSmith toggle

#	Section	Status	Location (Line)	Notes
8	Example Prompts	✓	L197	Table with 15 example prompts across 6 categories
9	Challenges Faced	✓	L261	5 challenges: tool routing loop, memory scope, RAG chunking, intent ambiguity, arg extraction
10	Future Improvements	✓	L275	8 improvements: multi-intent, auth, voice, proactive alerts, HITL, Docker, eval, i18n
11	Screenshots Section	✓	L221	Added — 7 screenshots with descriptions
12	Demo Video Section	✓	L235	Added — references demo_video.mp4

README Verdict: ✓ All 12 required sections present.

2. Workflow Diagram

Item	Status	Notes
<code>workflow_diagram.md</code> exists	✓	Standalone file with 4 Mermaid diagrams
Architecture overview diagram	✓	Flowchart: START → Classifier → 6 agents → Tool Executor → END
Tool mapping diagram	✓	Shows which tools belong to which agent
Data flow sequence diagram	✓	User → Gradio → Classifier → Agent → Tools → DB/Vector-Store → Response
Memory architecture diagram	✓	Session UUID → MemorySaver → turn-by-turn message history
Mermaid diagram in README	✓	Inline diagram at L45-68

Workflow Diagram Verdict: ✓ Complete with 4 diagrams across 2 files.

3. Screenshots

#	File	Content	Status
1	screenshots/ 01_initial_ui.png	Gradio UI on launch	✓
2	screenshots/ 02_bill_inquiry.png	Billing tool execution	✓
3	screenshots/ 03_plan_information. png	Plan listing tool exe- cution	✓
4	screenshots/ 04_technical_support _rag.png	RAG knowledge base search	✓
5	screenshots/ 05_memory_multiturn. png	Multi-turn memory demonstration	✓
6	screenshots/ 05b_conversation_con text.png	Context continuation across turns	✓
7	screenshots/ 06_usage_tracking.pn g	Usage tracking tool execution	✓

Required coverage:

| Requirement | Status | Screenshot(s) |

|-----|-----|-----|

| Gradio UI | ✓ | 01_initial_ui.png |

| Tool execution examples | ✓ | 02 , 03 , 04 , 06 |

| Example conversations | ✓ | 02 , 03 , 04 , 05 , 05b , 06 |

Screenshots Verdict: ✓ **7 screenshots covering all required categories.**

4. Core Application Components

4a. LangGraph Implementation

Item	Status	File	Notes
StateGraph used	✓	agents/graph.py	StateGraph(AgentState)
Nodes defined	✓	agents/graph.py	8 nodes: classifier, 6 agents, tools
Entry point set	✓	agents/graph.py	work-flow.set_entry_point("classifier")
Conditional edges	✓	agents/graph.py	3 add_conditional_edges calls
Graph compiled	✓	agents/graph.py	work-flow.compile(checkpointer=memory)

4b. Tools Implemented (minimum 3-5 required)

#	Tool Name	File	Category
1	get_bill_details	tools/billing.py	Billing
2	get_payment_history	tools/billing.py	Billing
3	get_outstanding_balance	tools/billing.py	Billing
4	list_available_plans	tools/plans.py	Plans
5	get_plan_details	tools/plans.py	Plans
6	compare_plans	tools/plans.py	Plans
7	get_usage_summary	tools/usage.py	Usage
8	get_daily_usage	tools/usage.py	Usage
9	get_customer_profile	tools/account.py	Account
10	look-up_customer_by_name	tools/account.py	Account
11	create_support_ticket	tools/account.py	Account
12	get_open_tickets	tools/account.py	Account
13	search_knowledge_base	tools/ tech_support.py	Technical (RAG)

Total: 13 tools (requirement: 3-5 minimum) ✓

4c. Memory Implementation

Item	Status	Notes
MemorySaver imported	✓	from lang-graph.checkpoint.memory import MemorySaver
Checkpointers applied to graph	✓	work-flow.compile(checkpointer=memory)
Session ID (thread_id) used	✓	UUID per session in app.py L82
Chat history persists across turns	✓	Full message list stored per thread
Clear chat resets session	✓	New UUID generated on clear

4d. Gradio Interface

Item	Status	Notes
gr.Blocks UI	✓	app.py L126
Chatbot component	✓	gr.Chatbot with copy button, avatar
Text input + Send button	✓	gr.Textbox + gr.Button
Example prompts	✓	10 example prompts via gr.Examples
Clear chat button	✓	Resets history and session ID
Custom CSS/theme	✓	gr.themes.Soft() + custom CSS

4e. Conditional Routing Logic

Item	Status	Notes
Intent classifier node	✓	LLM-based classification into 6 categories
<code>route_by_intent</code> function	✓	Maps intent → agent node name
Conditional edges from classifier	✓	Routes to <code>billing_agent</code> , <code>plans_agent</code> , etc.
<code>should_continue</code> function	✓	Decides tools vs END after agent response
Tool → agent re-routing	✓	Tool results routed back to originating agent

Core Components Verdict: ✓ All components implemented and verified.

5. Demo Video

Item	Status	Notes
<code>demo_video.mp4</code> exists	✓	649 KB, located at project root
Referenced in README	✓	Section at L235

Demo Video Verdict: ✓ Present.

6. Requirements.txt

Dependency	Status	Purpose
<code>langchain>=0.3.0</code>	✓	Core framework
<code>langchain-core>=0.3.0</code>	✓	Core abstractions
<code>langchain-openai>=0.2.0</code>	✓	OpenAI LLM/embeddings
<code>langchain-community>=0.3.0</code>	✓	Community integrations (Chroma, loaders)
<code>langgraph>=0.2.0</code>	✓	Graph-based agent orchestration
<code>openai>=1.30.0</code>	✓	OpenAI SDK
<code>chromadb>=0.5.0</code>	✓	Vector store for RAG
<code>gradio>=4.40.0</code>	✓	Web UI framework
<code>python-dotenv>=1.0.0</code>	✓	Environment variable loading
<code>langsmith>=0.1.0</code>	✓	Tracing (optional)

Requirements Verdict: ✓ All dependencies listed.

7. Bonus Features (Extra Credit)

Feature	Status	Notes
RAG with vector database	✓	ChromaDB + OpenAI text-embedding-3-small
Database integration	✓	SQLite with 5 tables, seeded data
LangSmith tracing	✓	Toggle via <code>LANG-CHAIN_TRACING_V2</code> env var
Streaming support	✓	<code>streaming=True</code> on Chat-OpenAI
Multiple tool categories	✓	5 categories across 13 tools

8. Evaluation Rubric Coverage

Criterion	Points	Status	Evidence
LangGraph workflow	20	✓	agents/graph.py — full StateGraph with 8 nodes
Tool integration	15	✓	13 tools across 5 files (requires 3+)
Code quality	15	✓	Clean structure, docstrings, type hints, comments
Gradio interface	15	✓	Full Blocks UI with examples, session mgmt
Conversation memory	10	✓	MemorySaver checkpointer with thread_id
API integration	10	✓	OpenAI Chat + Embeddings APIs
Conditional routing	10	✓	Intent classifier → 6 conditional paths
README/Documentation	10	✓	Comprehensive with all required sections
Demo video & screenshots	5	✓	7 screenshots + demo_video.mp4
Total	110		

Summary

Category	Status
README.md (all sections)	✓ Complete
Workflow Diagram	✓ Complete
Screenshots (7 images)	✓ Complete
Core Application	✓ Complete
Demo Video	✓ Present
Requirements.txt	✓ Complete
Bonus Features	✓ 5/5 implemented
Git Version Control	✓ 2 commits

● Overall Verdict: ALL REQUIREMENTS MET

No missing items detected. The project is ready for submission.